

Alternative parallel programming models: past , present and is there a future?

Laxmikant (Sanjay) Kale
University of Illinois at Urbana-Champaign

Alternative parallel programming models: past , present and is there a future?

Laxmikant (Sanjay) Kale University of Illinois at Urbana-Champaign

In this talk, I will describe my perspective as a researcher for 40 years on the topic of parallel programming models. Given the time constraints, I will only present a small sample of programming models including my group's, and focus on broader themes. Before MPI, there were parallel logic and functional languages, and then precursors to MPI like nxlib. These and the process of development of the MPI standard and its consequence will be discussed. Among the many alternative models that were developed, some were meant as competitors, some raised level of abstraction, or supported specialization. We will review a few models that survive today, including Charm++, Chapel, HPX and a few more. We will examine what the future programming model landscape may look like and what issues will shape them, including but not focusing solely on the needs of machine learning, data analytics or accelerators.

I will cover Historical and personal perspective

- I started undergrad in 1972, ~ when first microprocessor (Intel 4004) arrived
 - Also around when first “practical” mind machine **c.mmp** was built by Bill Wulf et al
 - Who gave a talk at IISC Bangalore around 1978, when I was a MS student there
- I did my PhD on Parallel Prolog (Logic Programming) in 1985
 - Much activity then, in part because of Japan’s Fifth Generation Computing project
 - Mid 1980s was a “Cambrian explosion” in parallel computing
 - Many later parallel computing researchers were part of that universe
 - Vipin Kumar, Ian Foster, Rusty Lusk, ...
- Other influences:
 - Rediflow project of Bob Keller et al – 1978
 - MultiLisp (Halstead et al)
 - Dataflow (Dennis, Arvind, ..)
- **Disclaimer: this presentation is from my subjective perspective and flawed memory**

Outline

- Briefly review early history of parallel programming
 - And its landscape of concepts
 - Parallel Functional languages, Logic Programming, Dataflow machines / languages
 - Identify concepts from those times that continue to influence today's ATMs
 - Process models -> chares, actors
- Genesis of MPI and landscape of similar forms
- What gave MPI its longevity?
- Which alternatives persisted and why
- Survey of some current Alternatives to MPI: (with varying level of detail)
 - Charm++, Dharma/vt, HPX, STAPL, Chapel, Legion, UPC ..
- Current scenario
- Comments about future

Imagined Parallel Machines (paper designs) or one of a kind implementations (e.g. Dataflow m/cs)

Multicomputer based parallel machines
C.Mmp (1972), Cm* , Concert (1982)

SIMD/Vector machines
Systolic machines

Network of

microprocessor based distributed memory parallel machines

Timing and availability of parallel machines had an impact on development of parallel programming models

Shared Memory Commercial machines
~1984 BBN Butterfly, Late 1980s: Multimax, Sequent

Clusters and Supercomputers

Parallel Functional Languages

$f(e1, e2)$

where $e1$ and $e2$ are expressions,
which may involve function calls

Parallelism: evaluate $e1$ and $e2$ in parallel.

Recursively.

evaluate $e1$, $e2$ as well as f (!) parallel.

$f(g(5, h(a, b), c), f1(f2(f3(x)))))$

$f(x) = \text{if } (x < 2) \text{ then } x \text{ else } f(x-1) + f(x-2)$

$f(x) = \text{if } ((x < 2), x, f(x-1) + f(x-2))$

Will be problematic if all 3
arguments are fully evaluated
before evaluating “if” --- won’t
terminate.

Allow killing ongoing parallel
evaluations (when their results are
not needed)

Or: create a dependency from condition. $(x < 2)$

“Let” statement (simplified)

$f(x, y) \leftarrow$

let $z = g(x)$, $t = g(y)$, $u = h(z, t)$

in $f1(z, u) + f2(x, t) + f3(x, z, t, u)$

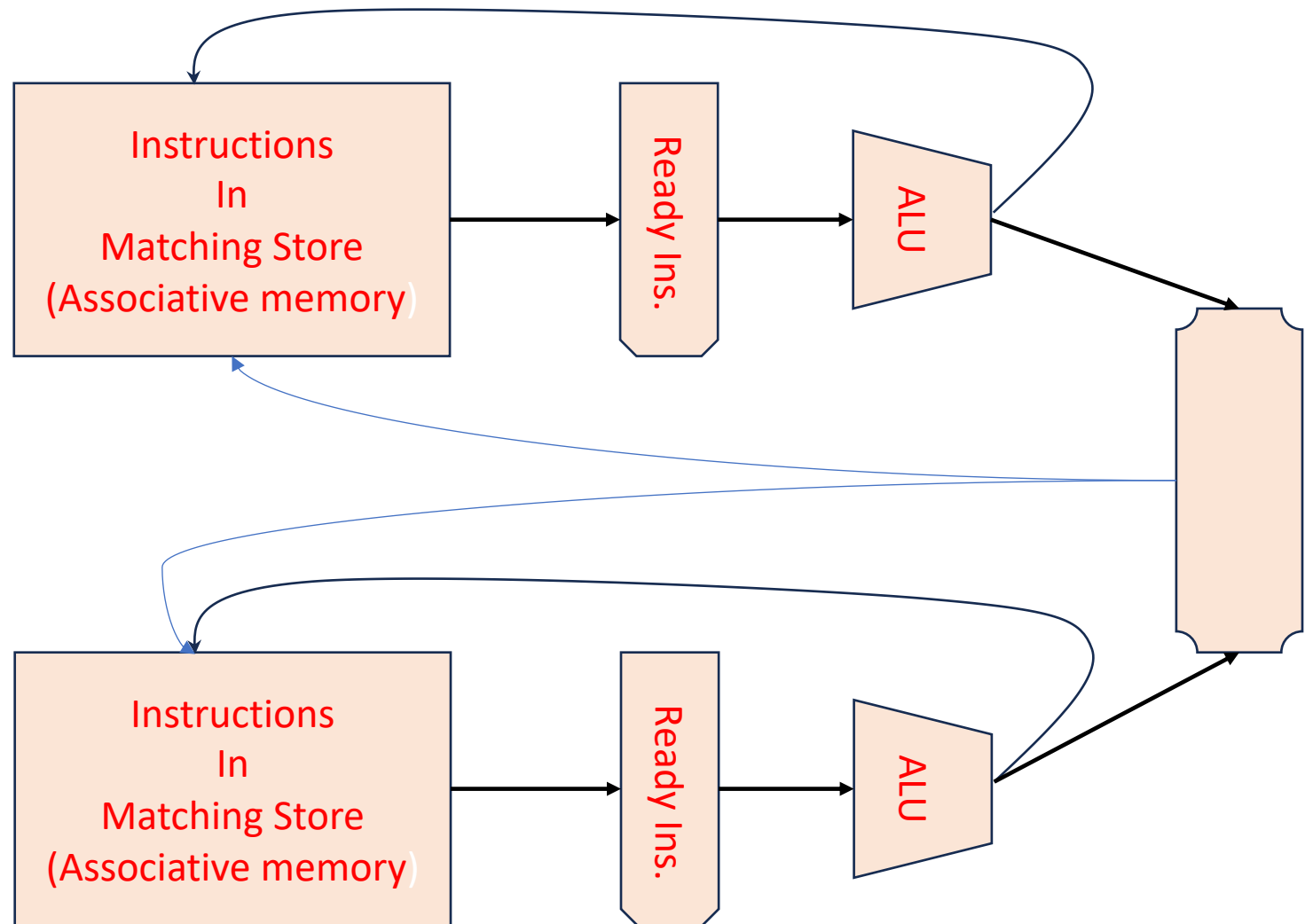
Leads to a set of **tasks**
with a **dependency graph** connecting them
And **data flowing** from tasks to tasks

Dataflow machines (caricatured) and lessons:

Data flow languages
implemented in parallel at
instruction level

For Functional Languages:
At each function call
“instantiate” and “launch”
instructions.

Ready instructions go to
a scheduler's queue.



The Matching Store was problematic: Expensive associative memory, Larger than expected

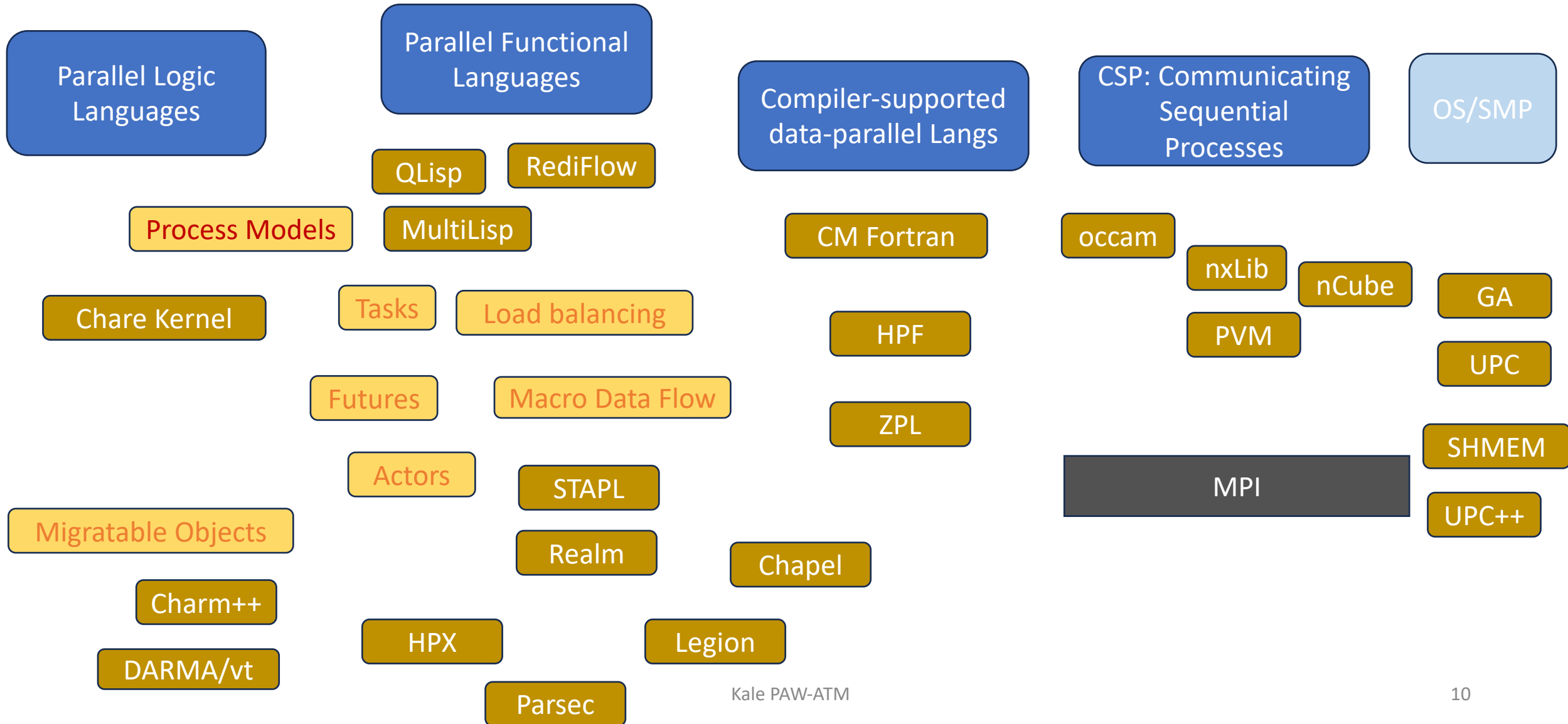
Dataflow machine Evolution

- I structures (to deal with arrays and loops), ..
- Monsoon project (again.. Simplified account)
 - Add “Activation Frame”
 - Analogous or same as Activation Record or stack frame
 - Except, not a linear stack: spaghetti stack (because of parallel function calls, ..)
 - Do not instantiate instructions in a matching store,
 - just create an Activation Frame with full-empty slots for each “variable”
 - Consult the code (static instructions. Only 1 copy) for what to do when a value arrives.
 - Fire (i.e. enqueue in ready queue) only instructions that are ready.
 - I.e. have both their operand values

Process Models

- Readiflow
 - functional programs
- Logic Programming
 - Aurora, And-Or Process Model, Reduce-Or P.M....
- Were not dependent on special machine like dataflow was
 - So the idea of activation record was natural for it. (invented/used w/o ado)
- Not instruction level
 - So, new tasks (e.g. function calls or logic program queries) and responses were implemented as messages between (logical) processes
- These logical processes:
 - Get created, have state, create others, get messages from children
 - Were naturally implemented as (what we'd now call) some sort of Actors

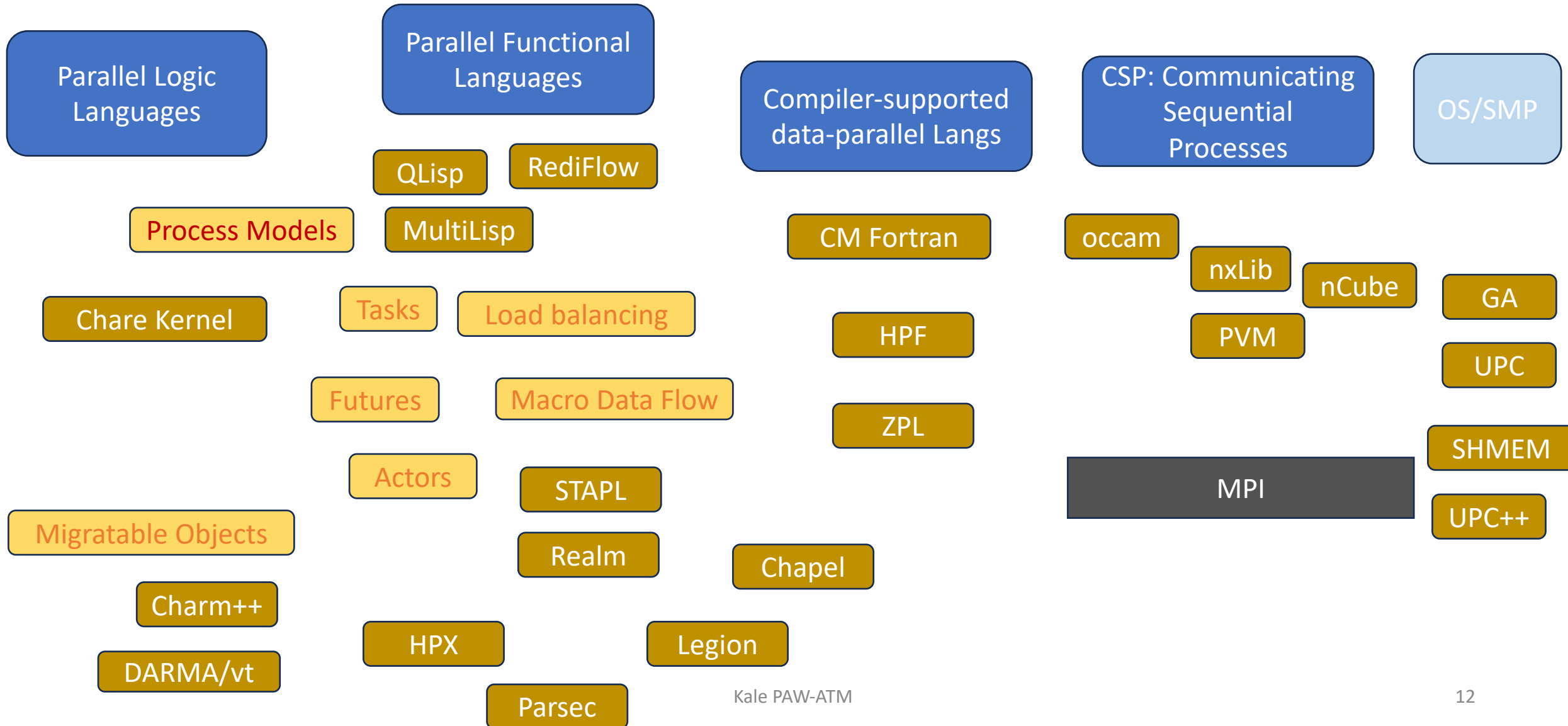
Simplified history as I recall



About MPI

- MPI set out to standardize common practice (nxlib, csp?, ncube, pvm, ..)
- But ended up with a design by committee.. was far more expansive
 - Worked for a good reason: the committee comprised vendors and users
 - So, buy-in and utility-oriented practical influence
 - That is the reason it survived and thrived
- Personally, I stayed away after engaging a bit early on
 - But found 2 things which I really did not like
 - Blocking collectives → fixed much later
 - Tag-matching and in-order delivery → sort of getting addressed but not adequately
 - We ended up using lower-level layers than MPI and supporting our own async collectives
- Its success inhibited innovation, although it was not intended to
 - New models should have been built on top of MPI

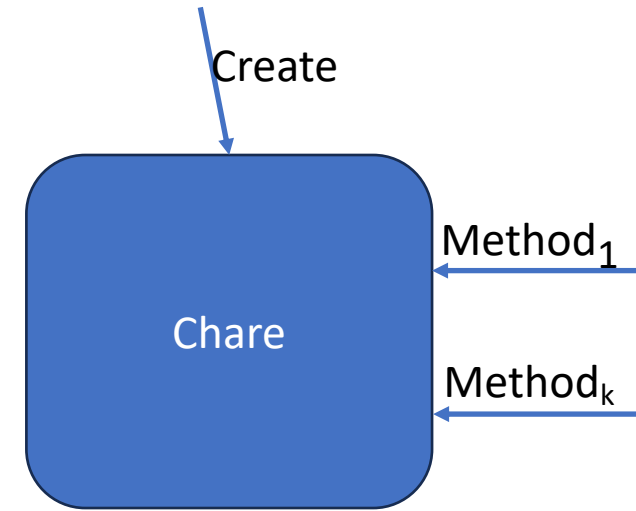
Parallel Functional and Logic programming :



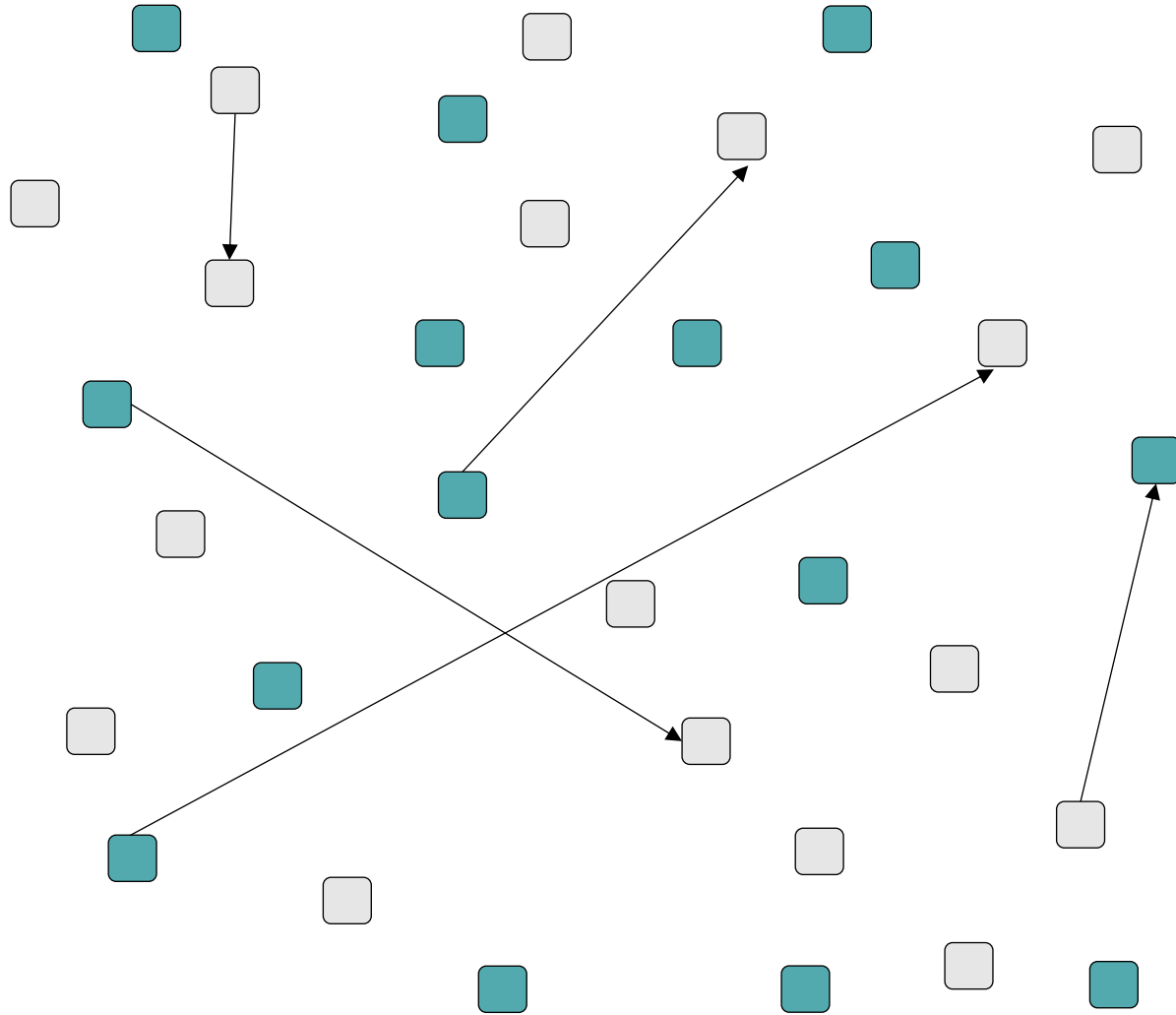
Charm++

Chare: A message-driven Object

- We saw the idea emerge with the “process models”
- Specifically, in C++ based Charm++
 - A “chare” is a C++ object
 - Can only access local data
 - Plus: readonly data, ...
 - Has some methods designated as **entry** methods
 - These can be invoked asynchronously, from remote objects
- There are typically many chares on a processor
 - And many seeds for new chares that can be moved freely
- With these chares, one can support:
 - Divide and Conquer algorithms
 - Combinatorial search
 - Functional and Logic languages



- A Charm++ computation consists of multiple collections of globally visible objects
- Each collection is individually indexed



Processor 0

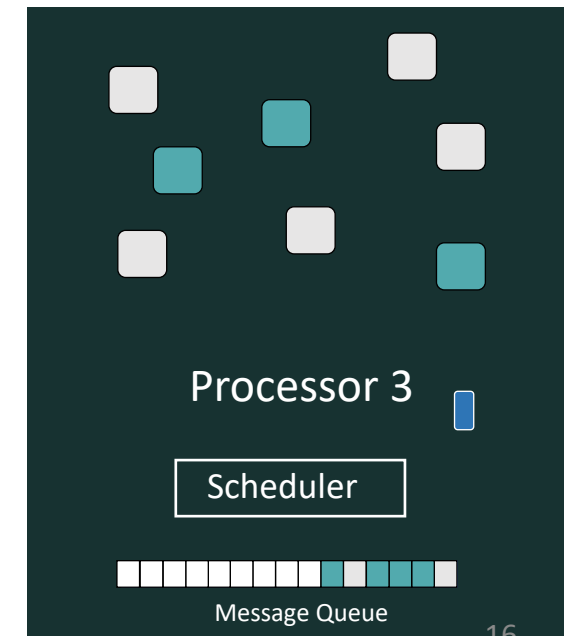
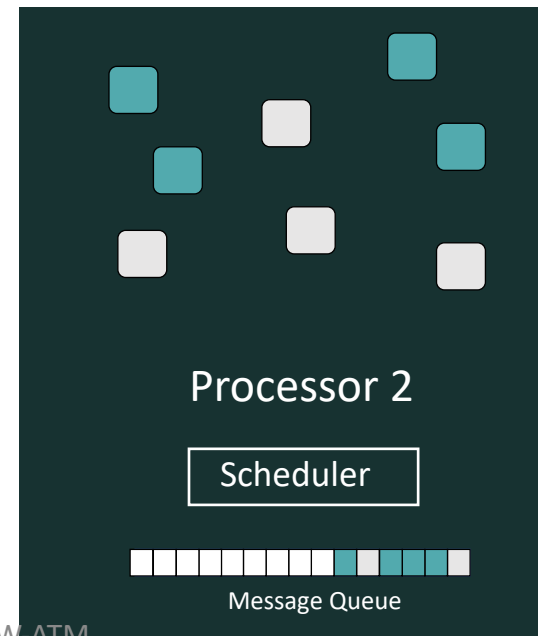
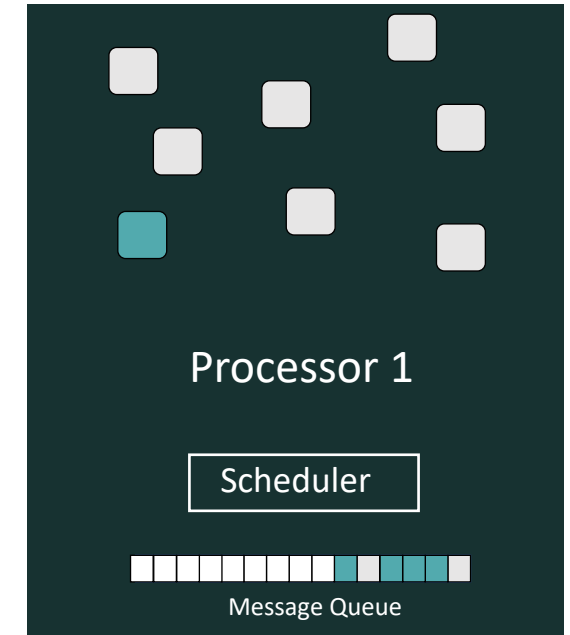
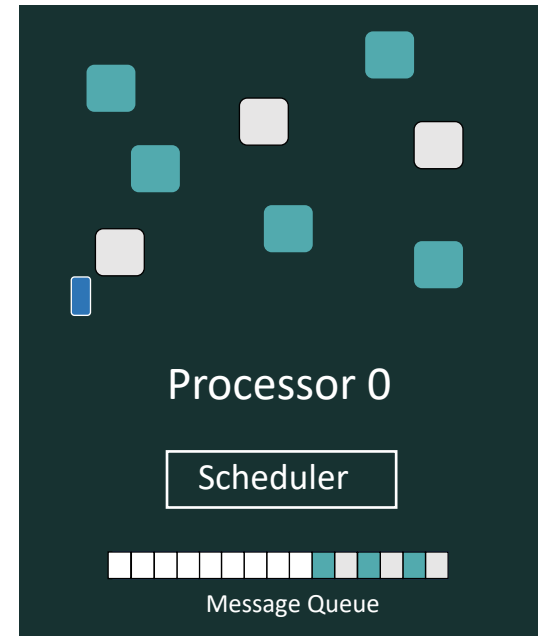
Processor 1

Processor 2

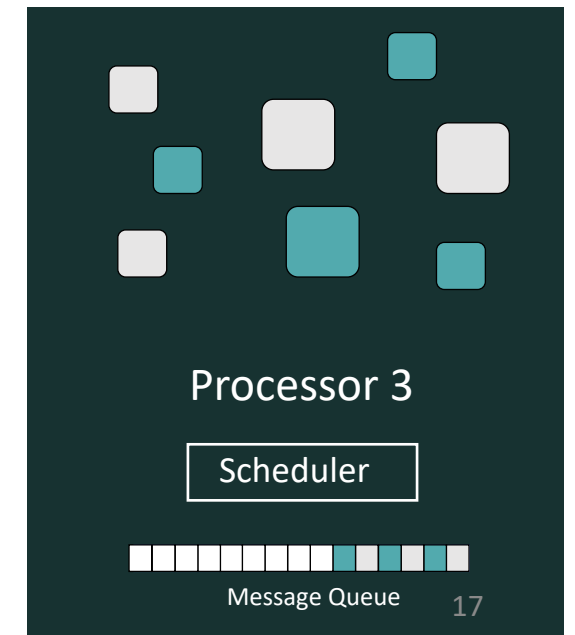
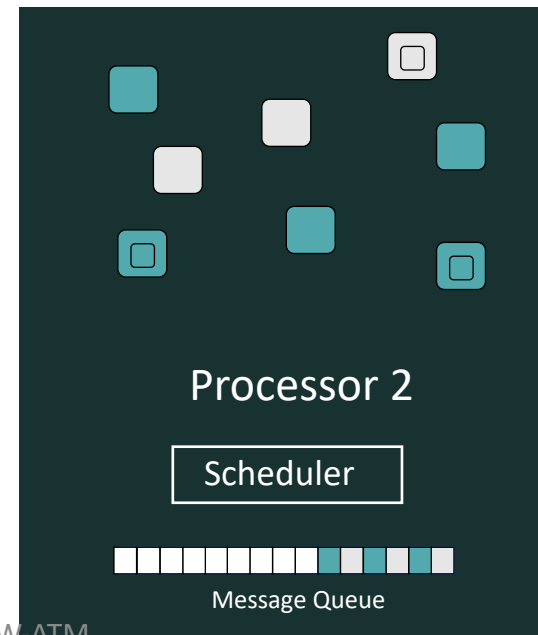
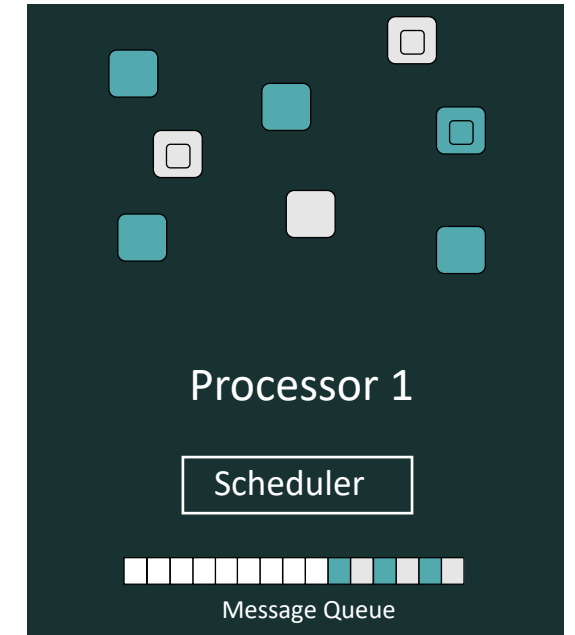
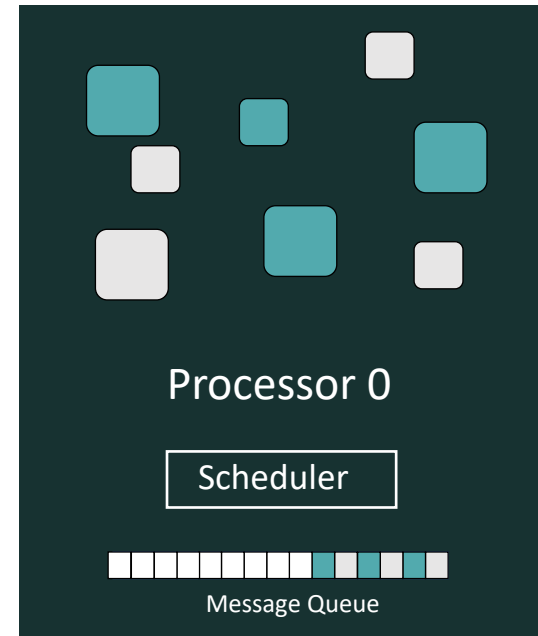
Processor 3

- A Charm++ computation consists of multiple collections of globally visible objects
- Each collection is individually indexed

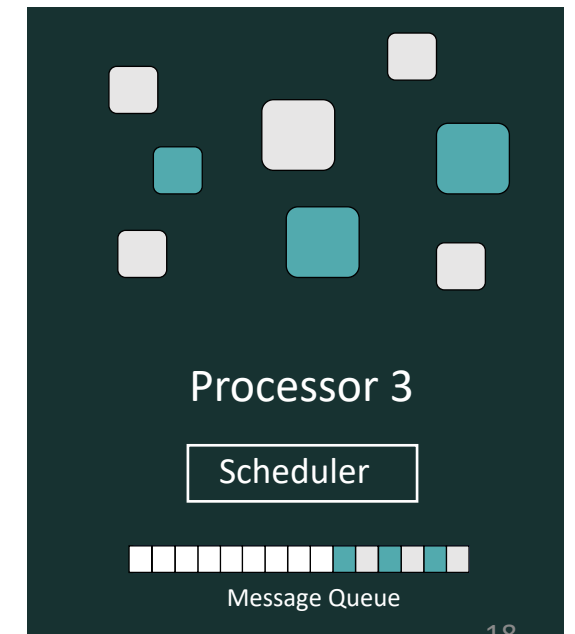
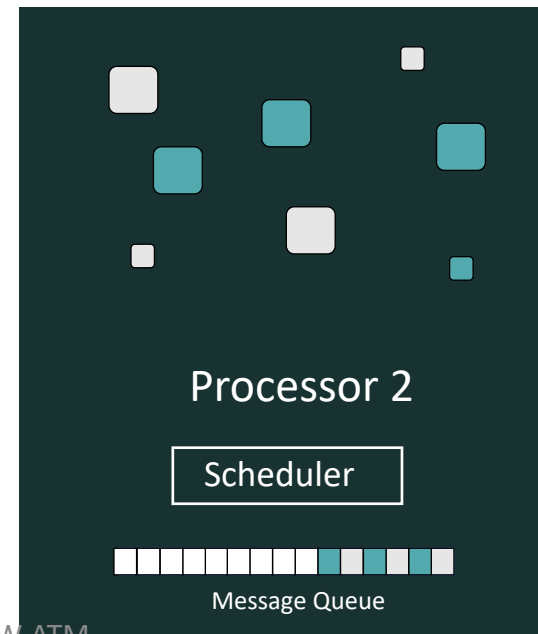
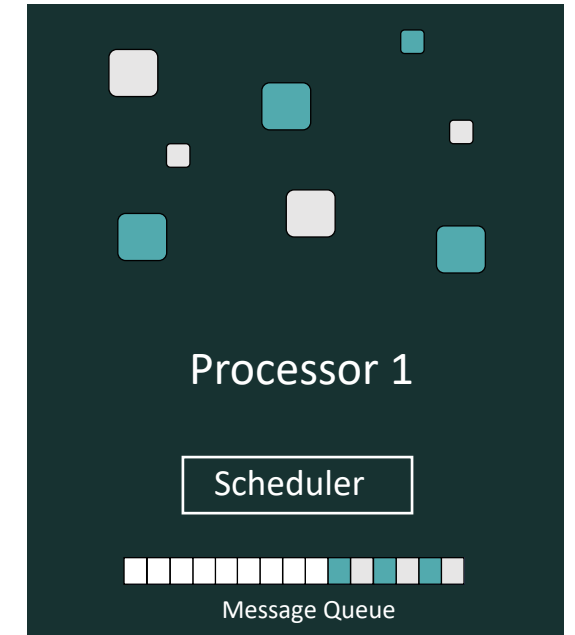
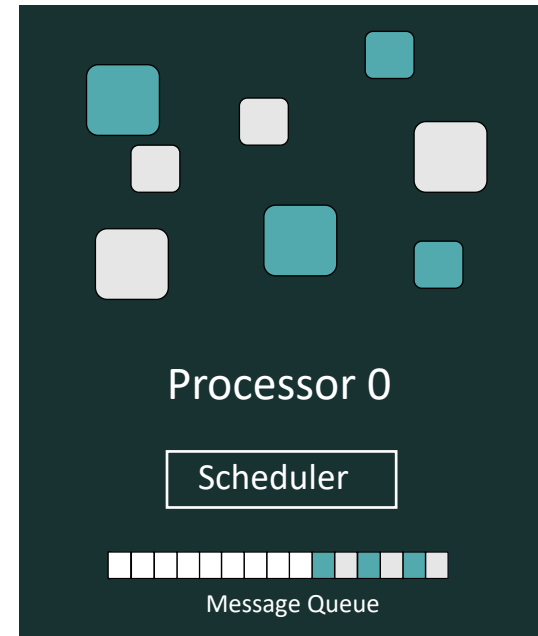
- Objects are assigned to processors by the runtime system
 - Programmer does not need to know where an object is located
- Scheduling on each processors is under the control of a user-space message-driven scheduler

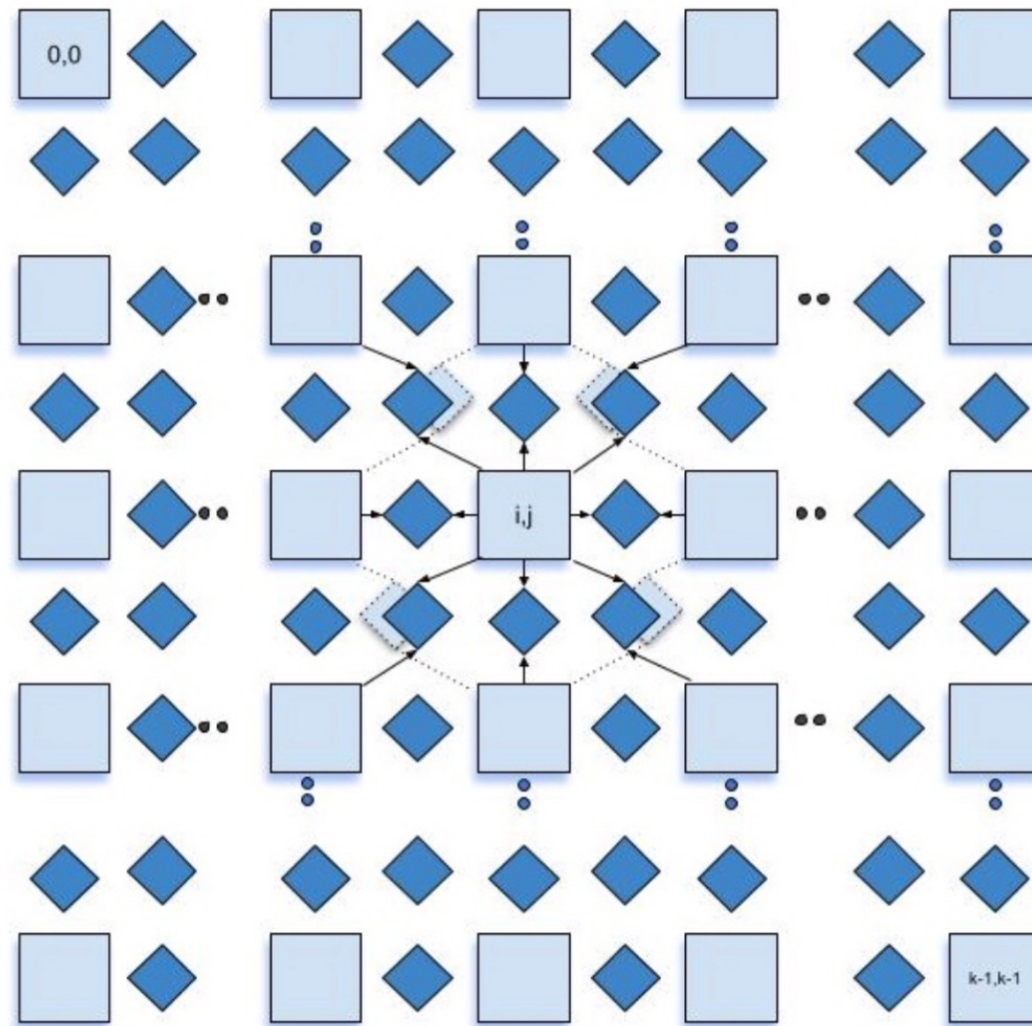


The runtime system knows which processors are overloaded, which objects are computationally heavy, which objects talk to which



Using this information, it migrates objects to rebalance load and optimize communication





LeanMD in 2 dimensions, with a $k \times k$ array of cell
charges and a $k \times k \times 3 \times 3$ sparse array of pair charges

Capabilities

- Capabilities

- Dynamic load balancing
- Fault Tolerance
- Elasticity:
 - change the set of nodes allocated to a job
- Adaptive overlap of communication and computation*
- Communication optimizations
- Out-of-core execution
- Energy optimizations*
- Asynchronous GPGPU interface

Demo on raspberry pi cluster:

<https://www.hpccharm.com/demo>

- Programming Systems

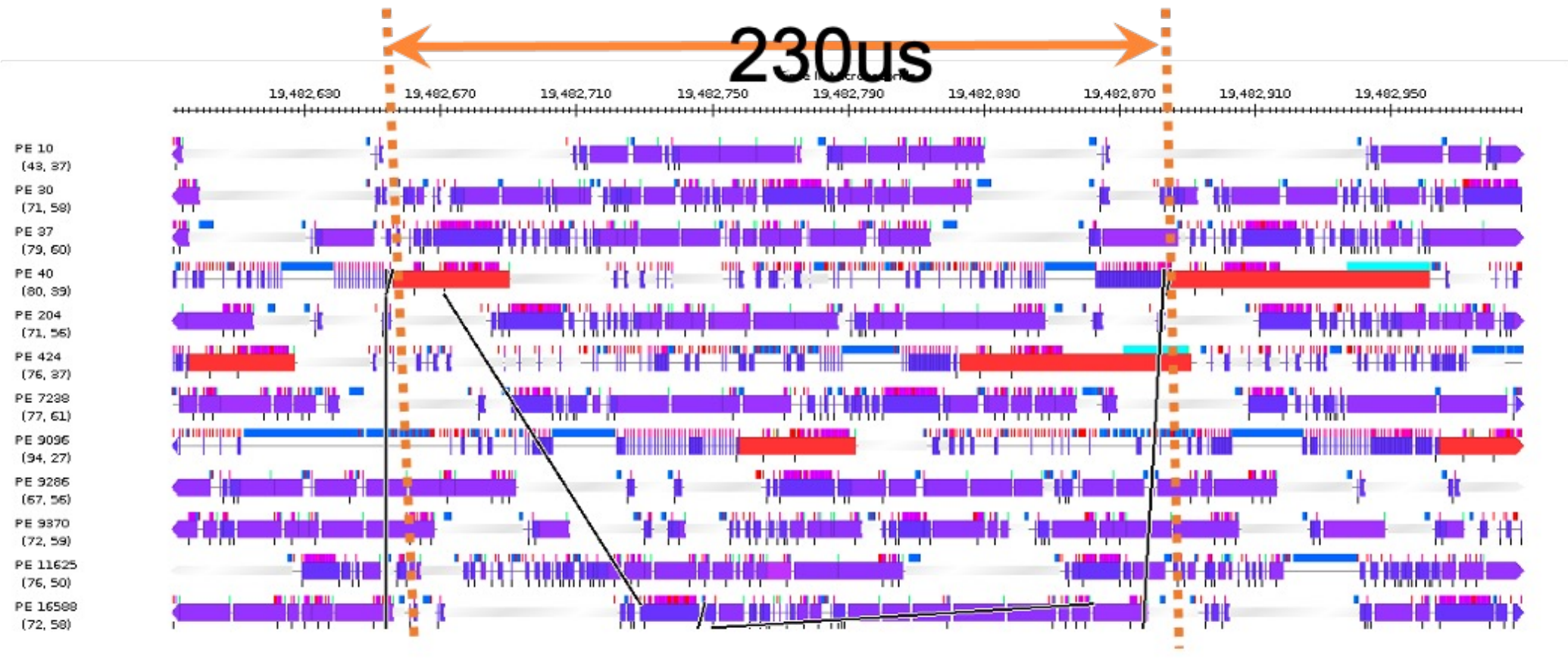
- Charm++
- Charm4Py
- Adaptive MPI
- Charisma, MSA< Divcon, ..
- CharmTyles

- Applications:

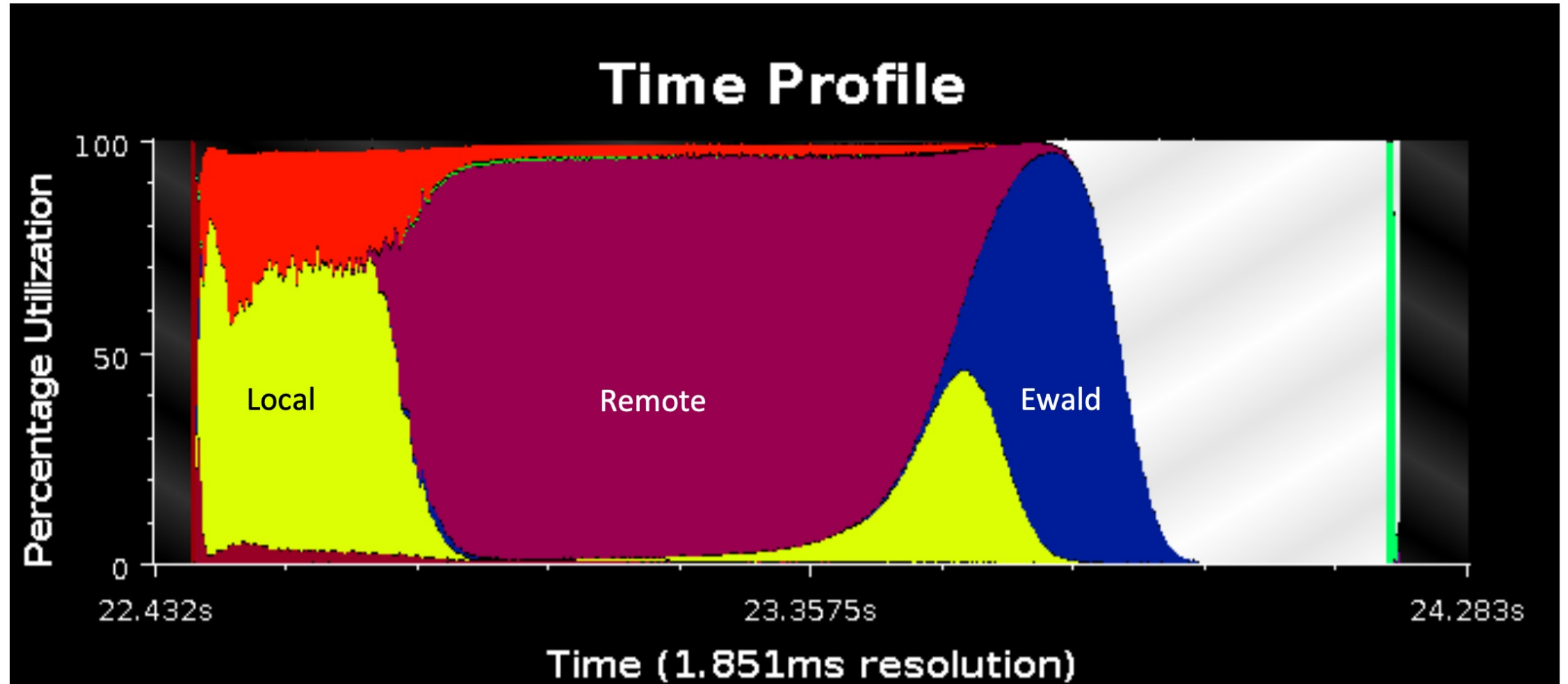
- NAMD (biophysics)
- ChaNGa (astronomy: gravity/sph)
- Quinoa (CFD)
- SPeCTRE (Astro:)
- Enzo (Astro: structured AMR)
- OpenAtom (elec. Structure)
- ..

More info and papers:

<https://charm.cs.Illinois.edu>



Example: NAMD Timeline on a small subset of processors in a run on 527 nodes (about 16000 cores) on IBM® Power7-based PERC machine.
 Illustrating Fine-grained tasks



Example: ChaNGa time profile showing activities over time on a 8K core run on Blue waters

Illustrating use of priorities and load balancing

DARMA/vt (SNL/ Lifflander et al)

- In some ways, a follow-on to Charm++
 - Jonathan Lifflander former student
 - DARMA preexisting project
- Main point of departure:
 - Make less friction for MPI users to adopt it
 - Strongly Co-Exist with MPI
 - And with Kokkos
- Only manage overdecomposed objects in specific modules
- Rest remains in MPI

STAPL

- Motivation/ signature idea: provide STL like interface for parallel programmers
- AP stands for Adaptive Parallel
- pAlgorithms, pContainers, pView
- pContainers: distributed data structures
- pView: segmented access (e.g. to a sub-tile of a matrix)
- Scaled to large machines such as Blue Gene
- Many projects to build upon this, such as graph algorithms

HPX collage

- future

```
std::uint64_t fibonacci(std::uint64_t n)
{
    if (n < 2)
        return n;

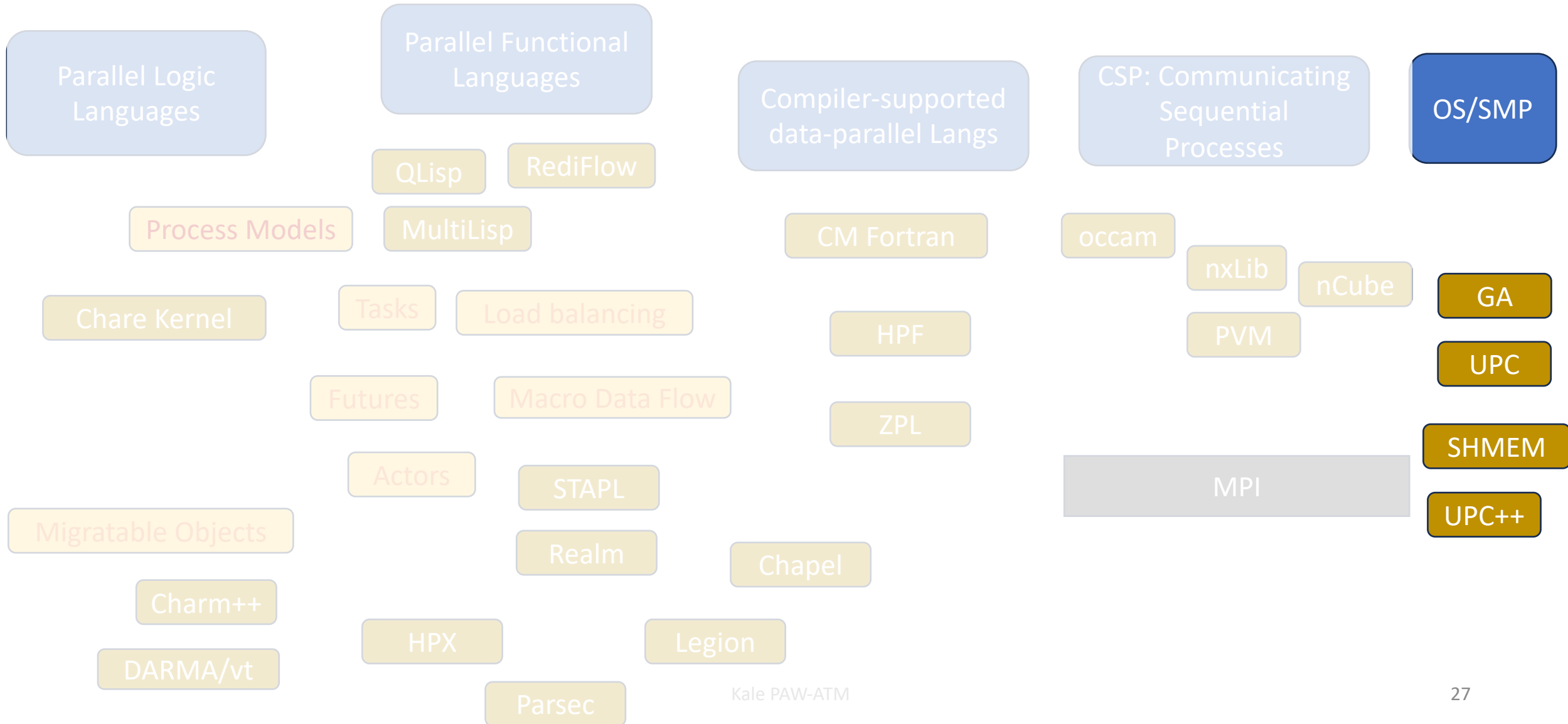
    hpx::future<std::uint64_t> n1 = hpx::async(fibonacci, n - 1);
    std::uint64_t n2 = fibonacci(n - 2);

    return n1.get() + n2;    // wait for the Future to return their values
}
```

HPX main appeal: C++ standard

- Prof. Kaiser and others have persistently engaged with C++ standards groups to push for parallel constructs
 - Aligned with HPX concepts
- Task based local runtime
- STL like extensions to C++: parallel algorithms
- Ability to spawn remote asynchronous actions by wrapping function calls in “action”
 - Equivalent of handlers in Converse or active messages, but with C++ parameter marshalling support
- Components: C++ objects with remotely invokable methods
 - Exposed by registering them as actions
- Active Global Object Space: Location management of global pointers (GIDs)

Global Shared Address Space



Shared Memory Programming

- Symmetric multiprocessors came to market in mid-late 1980s
 - Sequent Symmetry, Multimax Encore – 8-ish processors (cores in today's lingo)
 - Previous experience with BBN butterfly, ..
 - Programming: Pthreads, locks, memory fences..
 - Vendor-specific compiler directives (parallel loops)
 - Led to OpenMP
- Obstacles:
 - Race conditions: many chances for threads to interleave execution in different orders.
 - Performance: data produced by one pthread (on one core) used by another
 - Leads to cache misses and loss of performance.
 - Unintended data sharing: e.g. false sharing via cache lines,
 - So minimize that (e.g. tiled data structures)

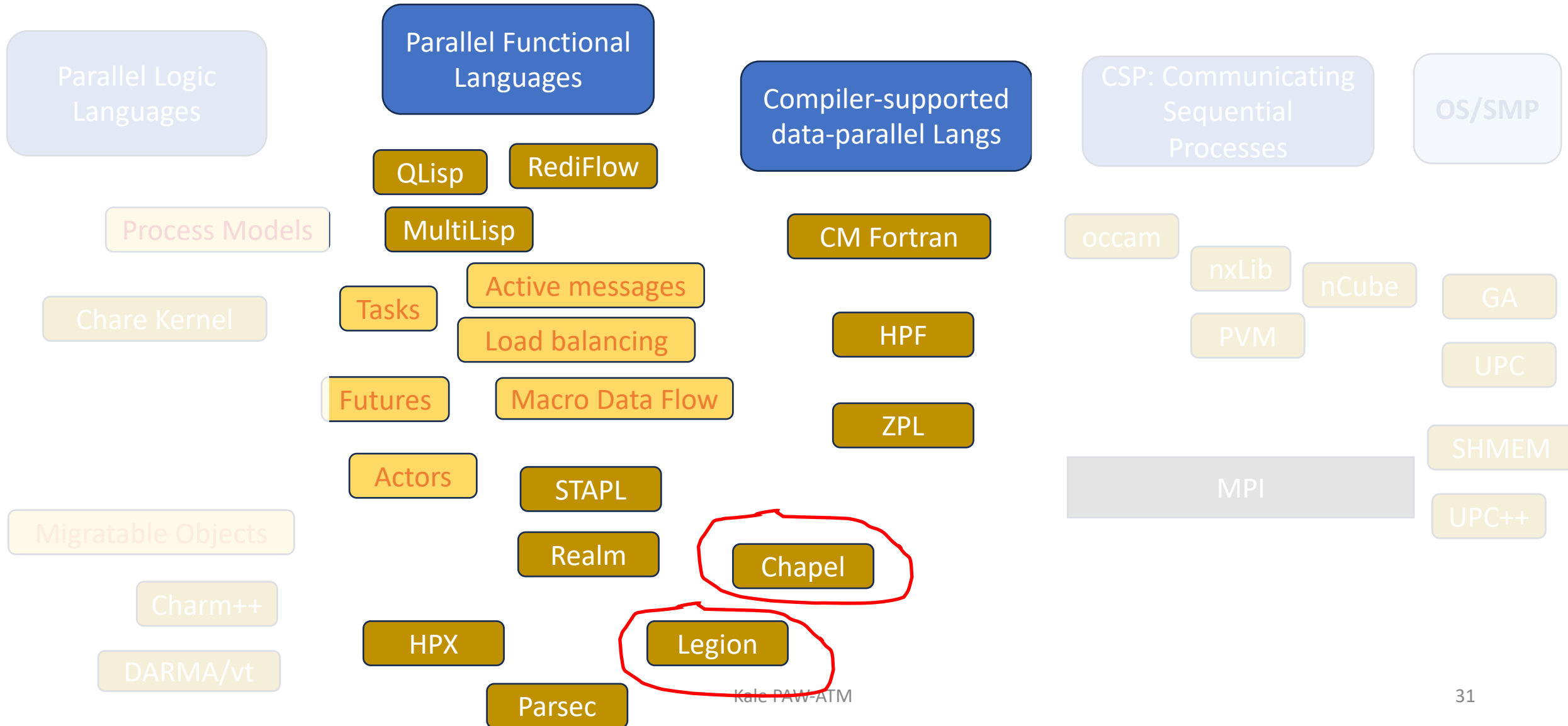
Global Address Space

- Some people who were programming shared memory machines felt that it is a simpler paradigm compared with message passing
- So, build abstractions similar to shared memory programming but that work on large distributed memory
- Main concerns: locality and race conditions
- As an aside:
 - Work by Jay Hoeflinger (w David Padua, ..) , as I recall
 - Took a shared memory program which (even on 1 node) didn't perform as well as MPI program)
 - and modified it in stages to match the performance
 - By the end, it started resembling the distributed memory program
- Idea: try to get some simpler consistency model and avoid item by item load-store

Partitioned Global Address Space Models

- Global Arrays (Jarek Nieplocha) 1994
 - Built on ARMCI one-sided communication library
 - Get or put for contiguous chunks of arrays
- UPC: Unified Parallel C (global pointer)
- CAF (Co-Array Fortran): For each array or scalar, an additional subscript specifies which processor's version of that array to is meant
 - Simplicity. Incorporated in later Fortran standards
- SHMEM and OpenSHMEM
- UPC++
 - C++ based, asynchronous communications

Compiler Supported Systems



Legion

- Realm: A tasks and events runtime
- Legion, basic ontology:
 - Data “tables” (think arrays of structs or structures of arrays)
 - Data regions
 - Same region can have multiple views : e.g. can do ghosts in stencil computation with shifted views
 - Data-driven tasks

Chapel

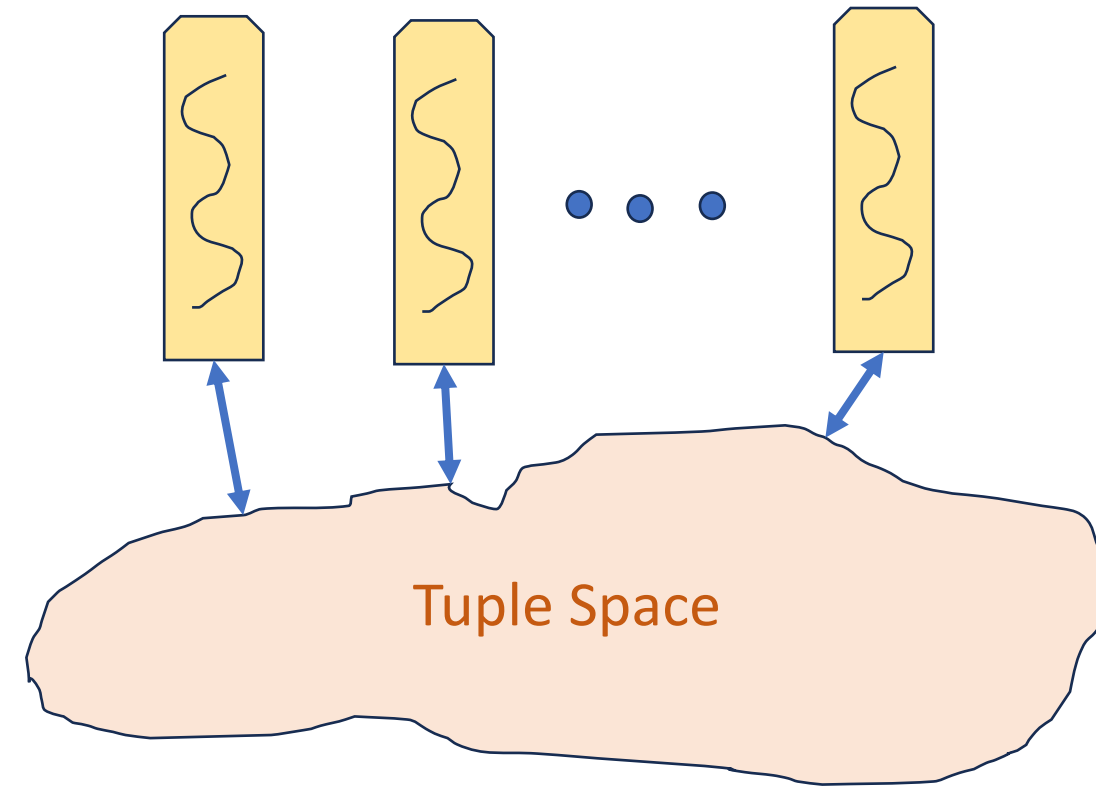
- In some ways, it follows in sequence of Compiler-supported Data parallel approaches such as HPF (High Performance Fortran), ZPL
- But it treads a unique path
 - Supports both data parallelism and task parallelism
 - Global or local view of data, control
 - Cobegin, Coforall loops
 - Locality aware distributed memory programming
 - Locales (e.g. node/rank), remote actions
 - “On” statement
 - Run task on a specified locale
 - Distributed arrays, forall

Which models survived and why

- Two main drivers, in addition to elegance and utility of the ideas
 - A long-term passionate champion
 - One (or preferably more) important application
- Two (sad) examples:
 - Linda (David Gelernter)
 - GA i.e. Global Arrays (Jarek Nieplocha)
- Other helpful attributes
 - Interoperation with MPI and with each other
 - Software sustainability
 - Need to collaborate : Coopetition
 - Componentization

Linda

- Remember Cambrian explosion analogy?
- Illustrates how a small number of primitives can lead to very different parallel prog model
- Basic concept: a set of processes interacting with a global “tuple space
- C-Linda: C as the base language
 - Only adds a couple of library calls:
 - `in(t)`, `read(t)` : find a tuple that matches a specified pattern.
 - `out(t)`
 - `Eval(args)`



Domain Specific Languages and Frameworks

- Another way in which ATMs can thrive
- Languages: compiler support, Framework: library
- Specialized high level languages matched for programming patterns
 - Charisma (static iterative data flow), MSA
 - Need to interoperate
- Domain Specific Languages and frameworks
 - cuPyNumerics, charmNumerics, ..
 - Combinatorial Search
 - Ray, Pytorch, ..
 - Dask, champandas, ...: parallel dataframes
 - Graph algorithm packages

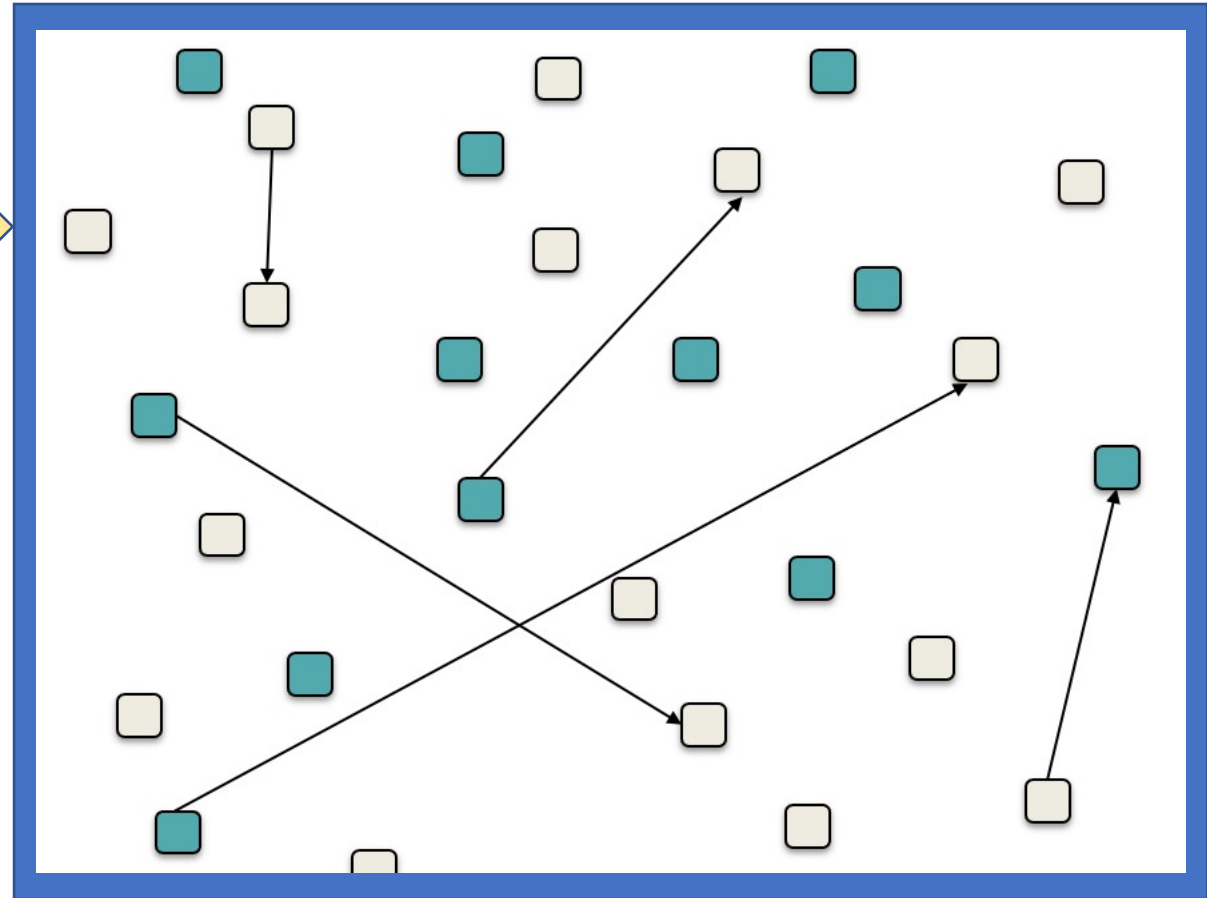
These combined with MPI, LCI or one of the general purpose ATMs may be a good future

CharmTyles: Python Interface for Charm++

Charm++ objects spread over a cluster &
Domain Specific libraries

Front-End
Python or
Domain-Specific Language,
Jupyter notebook

Intend to support a variety of
abstractions



Front-End:

Support convenient syntax and interface
Create and process ASTs

Centralized and Distributed Coordination/Orchestration

Parallel Domain-Specific
Libraries

Just-in-time compilation
(Kernels, ..)

Dynamic creation of Object
Collections, and DAGs
encoding data/control
dependences

Parallel Cluster

Data Sources/Sinks

Web-based, cloud-based as well as cluster's parallel file system based

CharmTyles Target Domains and Status

- Numerical Linear Algebra
 - Initial prototype numpy-like implementation demonstrated
- Stencil Computations
 - Simple Prototype exists. Compiler optimizations needed
- Dataframes (Pandas, ..):
 - Initial implementation based on apache arrow available
- Graph Algorithms:
 - Distributed Data structure developed, SSSP demonstrated
- [Spatial] Tree algorithms: plan to leverage paraTreeT framework
- ...

Look on github for charmnumerics, libcharmtypes, charmpandas
The org is UIUC-PPL but may change to charmplusplus

Resource Elasticity

- We expect to demonstrate all of the capabilities described above in a mode where the number of nodes used by the application can be changed at runtime in many cloud-based-cluster environments
 - Kubernetes prototype has been demonstrated
 - SC workshop presentation by Aditya Bhosale (Canopie)
 - Production use being enhanced
 - AWS and other systems

Conclusion

- There have been alternatives to MPI since the early days
 - And some survive and “thrive” today... old and new
- They build upon a collection of common themes and abstractions
 - Tasks, dependences, asynchronous execution, rdma, handlers, load balancing, location management, futures, compiler support, data parallelism, partitioned global address space, ...
- Higher level, rather than general purpose, may see more activity in future
- A case for interoperation and coopetition
 - If we are to succeed, we must collaborate, possibly on creating shared software components for common functionalities